

Dasar Database Relasional: Preparation, DDL, DML, dan Query CRUD PostgreSQL

Durasi: 3 Jam (180 Menit) **Level:** Intermediate (Menengah)

Sebuah aplikasi backend tidak dapat dipisahkan dari media penyimpanan data persisten. Di dunia pengembangan perangkat lunak modern, database relasional (**RDBMS**) seperti **PostgreSQL** adalah pilihan utama untuk industri finansial, e-commerce, dan enterprise karena keandalannya dalam menjaga integritas data (kepatuhan *ACID*).

Modul ini akan membedah persiapan instalasi PostgreSQL, struktur arsitektur database relasional, cara mendefinisikan tabel menggunakan SQL **DDL (Data Definition Language)**, serta cara mengelola dan memanipulasi data transaksi menggunakan SQL **DML (Data Manipulation Language)** untuk operasi CRUD dasar.

Sesi 1: Preparation, Setup, dan Struktur Relasional (45 Menit)

Sebelum menulis query SQL, kita perlu mempersiapkan lingkungan database PostgreSQL di komputer lokal kita.

1. Setup Database dengan Docker (Modern & Cepat)

Menggunakan Docker untuk database lokal sangat direkomendasikan karena menghindari konfigurasi OS yang rumit.

Jalankan perintah berikut di terminal Anda untuk mengaktifkan kontainer PostgreSQL:

```
# Menjalankan kontainer postgres dengan password 'secret123' di port 5432
docker run --name goshop-db -e POSTGRES_PASSWORD=secret123 -p 5432:5432 -d postgres
```

2. Koneksi Database via CLI (`psql`)

Masuk ke terminal kontainer PostgreSQL untuk menjalankan perintah CLI:

```
# Masuk ke CLI psql sebagai superuser 'postgres'
docker exec -it goshop-db psql -U postgres
```

Perintah dasar (*slash commands*) di `psql` :

- `\l` : Menampilkan daftar semua database.
- `\c <nama_db>` : Berpindah koneksi ke database tertentu.
- `\dt` : Menampilkan semua tabel di skema saat ini.
- `\d <nama_tabel>` : Melihat detail kolom dan tipe data dari suatu tabel.
- `\q` : Keluar dari terminal `psql` .

3. Struktur Dasar RDBMS

Database relasional mengorganisasikan data dalam bentuk tabel dua dimensi yang terdiri dari:

- **Table (Entitas)**: Kumpulan data sejenis (misal tabel `products`).
- **Columns (Attributes/Fields)**: Karakteristik data (misal `name` , `price` , `stock`).
- **Rows (Records/Tuples)**: Baris data riil yang disimpan.
- **Primary Key (PK)**: Kolom kunci unik pengidentifikasi baris data (tidak boleh null, tidak boleh duplikat).
- **Foreign Key (FK)**: Kolom yang menghubungkan satu tabel ke tabel lainnya untuk membangun relasi.

Sesi 2: SQL DDL (Data Definition Language) (45 Menit)

DDL digunakan untuk membuat, mengubah, dan menghapus struktur objek database (seperti tabel, skema, dan indeks).

1. Pembuatan Tabel & Tipe Data PostgreSQL

Saat mendefinisikan tabel, pemilihan tipe data yang tepat sangat menentukan efisiensi ruang penyimpanan dan kecepatan indeks:

- `SERIAL` / `BIGSERIAL` : Integer auto-increment (cocok untuk ID sederhana).
- `UUID` : Identifier unik universal 128-bit (sangat aman untuk sistem terdistribusi/microservices).
- `VARCHAR(n)` : Teks dengan panjang dinamis maksimal `n` karakter.
- `TEXT` : Teks panjang tanpa batasan karakter.
- `NUMERIC(p, s)` / `DECIMAL(p, s)` : Angka desimal presisi tetap. Sangat wajib digunakan untuk **data keuangan/harga** (misal `NUMERIC(15, 2)`). Jangan gunakan float untuk uang!
- `TIMESTAMP` : Informasi tanggal dan waktu.

2. Constraints (Batasan Data)

Constraints menjamin validitas data sebelum ditulis ke disk:

- `NOT NULL` : Kolom wajib diisi.

- **UNIQUE** : Nilai kolom tidak boleh kembar di tabel tersebut.
- **DEFAULT <nilai>** : Memberikan nilai otomatis jika kolom dikosongkan saat insert.
- **CHECK (<kondisi>)** : Memvalidasi aturan logika (misal harga tidak boleh kurang dari nol).

3. Contoh Query DDL Relasional (One-to-Many)

```
-- 1. Membuat tabel Kategori
CREATE TABLE categories (
  id SERIAL PRIMARY KEY,
  name VARCHAR(50) NOT NULL UNIQUE
);

-- 2. Membuat tabel Produk yang merujuk ke Kategori
CREATE TABLE products (
  sku VARCHAR(50) PRIMARY KEY,
  category_id INT NOT NULL,
  name VARCHAR(100) NOT NULL,
  price NUMERIC(12, 2) NOT NULL CHECK (price ≥ 0),
  stock INT NOT NULL DEFAULT 0 CHECK (stock ≥ 0),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  -- Foreign Key Constraint dengan aturan relasional
  CONSTRAINT fk_product_category
    FOREIGN KEY (category_id)
    REFERENCES categories(id)
    ON DELETE RESTRICT -- Mencegah kategori dihapus jika masih ada produk di
dalamnya
);
```

Sesi 3: SQL DML (Data Manipulation Language) (45 Menit)

DML digunakan untuk mengelola data di dalam tabel yang sudah didefinisikan oleh DDL. Operasi DML mewakili siklus hidup data: **Create, Read, Update, dan Delete (CRUD)**.

1. INSERT (Create)

Memasukkan baris data baru ke dalam tabel.

```
-- Insert dengan klausa RETURNING untuk mengambil data auto-increment secara instan
INSERT INTO categories (name)
VALUES ('Laptop')
RETURNING id, name;
```

2. SELECT (Read)

Membaca dan memfilter data dari database.

```
-- Membaca kolom spesifik dengan filter WHERE, pengurutan, dan paginasi (limit/offset)
SELECT sku, name, price, stock
FROM products
WHERE price > 1000000.00 AND stock > 0
ORDER BY price DESC
LIMIT 10 OFFSET 0;
```

3. UPDATE (Update)

Memperbarui data yang sudah ada di tabel.

```
-- Wajib menyertakan WHERE clause yang spesifik agar tidak mengubah seluruh isi tabel!
UPDATE products
SET price = 14500000.00, stock = stock + 5
WHERE sku = 'SKU-LAP-01';
```

4. DELETE (Delete)

Menghapus baris data dari tabel.

```
-- Menghapus data produk tertentu
DELETE FROM products
WHERE sku = 'SKU-LAP-01';
```

NOTE

DELETE vs TRUNCATE:

- **DELETE** : Menghapus baris satu per satu, mengevaluasi trigger, lambat untuk data besar, bisa dibatalkan (*rollback*).
- **TRUNCATE** : Mengosongkan seluruh tabel secara instan dengan mereset penyimpanan tingkat disk, sangat cepat, tidak bisa dievaluasi per baris.

Sesi 4: Praktikum Mandiri: "Skema Database & Query CRUD Toko Online" (45 Menit)

Mari kita satukan materi DDL, DML, Constraints, dan Relasional ke dalam skrip SQL siap uji coba untuk skenario kasir Toko Online.

Skrip SQL Praktikum Lengkap

Anda dapat menyalin query di bawah ini dan menjalankannya secara berurutan di PostgreSQL Client Anda (pgAdmin 4 Query Tool atau terminal `psql`):

```
-- =====
-- 1. CLEANUP SKEMA DATABASE (Jika tabel sudah ada sebelumnya)
-- =====

DROP TABLE IF EXISTS products;
DROP TABLE IF EXISTS categories;

-- =====
-- 2. DDL - MEMBUAT TABEL KATEGORI DAN PRODUK
-- =====

CREATE TABLE categories (
    id SERIAL PRIMARY KEY,
    name VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE products (
    sku VARCHAR(50) PRIMARY KEY,
    category_id INT NOT NULL,
    name VARCHAR(100) NOT NULL,
    price NUMERIC(15, 2) NOT NULL CHECK (price ≥ 0),
    stock INT NOT NULL DEFAULT 0 CHECK (stock ≥ 0),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT fk_product_category
        FOREIGN KEY (category_id)
        REFERENCES categories(id)
        ON DELETE RESTRICT
);

-- =====
-- 3. DML - MEMASUKKAN DATA (INSERT)
-- =====

-- Insert Kategori (Dapatkan ID 1 dan 2)
INSERT INTO categories (name) VALUES ('Gadget') RETURNING *;
INSERT INTO categories (name) VALUES ('Aksesoris') RETURNING *;

-- Insert Produk ke Kategori Gadget (ID: 1)
INSERT INTO products (sku, category_id, name, price, stock)
VALUES ('SKU-ROG-01', 1, 'Laptop Asus ROG', 25000000.00, 5) RETURNING *;
```

```

INSERT INTO products (sku, category_id, name, price, stock)
VALUES ('SKU-IPH-02', 1, 'iPhone 15 Pro', 18000000.00, 10) RETURNING *;

-- Insert Produk ke Kategori Aksesoris (ID: 2)
INSERT INTO products (sku, category_id, name, price, stock)
VALUES ('SKU-MOU-03', 2, 'Mouse Wireless Silent', 350000.00, 25) RETURNING *;

INSERT INTO products (sku, category_id, name, price, stock)
VALUES ('SKU-KEY-04', 2, 'Keyboard Mechanical RGB', 1200000.00, 0) RETURNING *; --
stok kosong

-- =====
-- 4. DML - QUERY MEMBACA DATA (SELECT)
-- =====
-- Uji 1: Ambil semua produk yang harganya di atas Rp 1.000.000 dan stoknya ready
SELECT sku, name, price, stock
FROM products
WHERE price > 1000000.00 AND stock > 0
ORDER BY price DESC;

-- Uji 2: Cari produk aksesoris (category_id: 2) yang stoknya kosong
SELECT *
FROM products
WHERE category_id = 2 AND stock = 0;

-- =====
-- 5. DML - QUERY UPDATE DATA (UPDATE)
-- =====
-- Uji 3: Ubah stok keyboard mechanical RGB (stok kosong menjadi restock 10)
UPDATE products
SET stock = 10
WHERE sku = 'SKU-KEY-04'
RETURNING sku, name, stock;

-- =====
-- 6. DML - QUERY HAPUS DATA (DELETE)
-- =====
-- Uji 4: Hapus produk iPhone 15 Pro dari database
DELETE FROM products
WHERE sku = 'SKU-IPH-02';

-- Uji 5: Verifikasi data akhir di tabel produk
SELECT * FROM products;

```

Sesi 5: Soal Latihan Praktik E-Commerce (Mandiri)

Selesaikan dua latihan database relasional di bawah ini untuk menguji pemahaman Anda mengenai DDL, constraints, relasi tabel, dan manipulasi DML.

Latihan 1: Skema Database Pelanggan & Alamat (DDL Relasional)

Deskripsi Soal: Sebuah e-commerce memerlukan skema database relasional untuk menyimpan profil pengguna (*Customers*) dan daftar alamat pengiriman mereka (*Shipping Addresses*). Seorang customer dapat memiliki lebih dari satu alamat pengiriman (Hubungan *One-to-Many*).

Kriteria Implementasi (Tulis Query DDL):

1. Buatlah tabel `customers` dengan ketentuan:
 - `id` : Tipe `VARCHAR(50)` sebagai Primary Key.
 - `name` : Tipe `VARCHAR(100)` , tidak boleh kosong.
 - `email` : Tipe `VARCHAR(100)` , tidak boleh kosong, dan harus **unik**.
 - `registered_at` : Tipe `TIMESTAMP` dengan nilai default waktu sekarang.
 2. Buatlah tabel `shipping_addresses` dengan ketentuan:
 - `id` : Tipe `SERIAL` sebagai Primary Key.
 - `customer_id` : Tipe `VARCHAR(50)` , tidak boleh kosong, berfungsi sebagai **Foreign Key** yang merujuk ke `customers(id)` .
 - `street` : Tipe `TEXT` , tidak boleh kosong.
 - `city` : Tipe `VARCHAR(50)` , tidak boleh kosong.
 - `postal_code` : Tipe `VARCHAR(5)` dengan constraint `CHECK` untuk memastikan panjang karakter pos tepat 5 digit angka.
 - Atur aturan Foreign Key: **ON DELETE CASCADE** (Jika baris data customer dihapus, semua alamat pengirimannya otomatis ikut terhapus secara berantai).
-

Latihan 2: Query Analisis Transaksi Kasir (DML CRUD)

Deskripsi Soal: Setelah sistem checkout berhasil memproses belanjaan, data disimpan ke dalam tabel transaksi. Anda diminta menyusun query DML untuk mencatat, memperbarui, dan menganalisis data transaksi tersebut.

Asumsikan Struktur Tabel `orders` :

- `id` (VARCHAR), `customer_id` (VARCHAR), `total_amount` (NUMERIC), `payment_status` (VARCHAR), `order_date` (TIMESTAMP).

Kriteria Implementasi (Tulis Query DML):

1. Tulis query `INSERT` untuk memasukkan 3 baris transaksi baru:
 - Transaksi 1: Total Rp 2.500.000, Status: `'PAID'` , Customer: `'CUST-01'` .

- Transaksi 2: Total Rp 12.000.000, Status: 'PENDING', Customer: 'CUST-02'.
 - Transaksi 3: Total Rp 450.000, Status: 'FAILED', Customer: 'CUST-01'.
2. Tulis query `SELECT` untuk menampilkan semua transaksi yang berhasil lunas ('PAID') dengan nilai belanja di atas Rp 1.000.000, diurutkan dari transaksi terbesar ke terkecil.
 3. Tulis query `UPDATE` untuk mengubah semua transaksi yang berstatus 'PENDING' menjadi 'FAILED' jika transaksi tersebut dibuat pada hari kemarin (simulasikan dengan menyaring ID transaksi tertentu).
 4. Tulis query `DELETE` untuk membersihkan semua transaksi yang gagal bayar ('FAILED') untuk mengurangi memori sampah database.